

```
krishnamohan_koyya@instance-1:~$ kubectl get services
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kafka	NodePort	10.102.146.233	<none>	9092:30092/TCP	51d
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	84d
mysqldb	ClusterIP	None	<none>	3306/TCP	51d
ums-add-service	LoadBalancer	10.109.157.19	10.109.157.19	8080:31798/TCP	51d
ums-find-service	LoadBalancer	10.97.147.154	10.97.147.154	8080:31012/TCP	51d
ums-search-service	LoadBalancer	10.101.175.196	10.101.175.196	8080:30538/TCP	29d
zookeeper	NodePort	10.96.19.112	<none>	2181:30181/TCP	51d

Figure 2: List of services on the cluster

```
krishnamohan_koyya@instance-1:~$ curl -X POST -H 'Content-Type: application/json' -i http://10.109.157.19:8080/user
--data '{"name":"Krishna Mohan Koyya", "phone":9731423166}'
HTTP/1.1 200
Content-Type: application/json
Transfer-Encoding: chunked
Date: Fri, 02 Sep 2022 13:03:54 GMT
```

Figure 3: Adding user

```
krishnamohan_koyya@instance-1:~$ kubectl exec -it ums-journal-service-5f7f8fb4d5-4tm4g -- bash
root@ums-journal-service-5f7f8fb4d5-4tm4g:/# cat audits.log
Fri Sep 02 12:57:07 UTC 2022 name=Krishna Mohan, phone=9731423166, since=Fri Sep 02 12:57:05 UTC 2022
Fri Sep 02 13:02:48 UTC 2022 name=Krishna Mohan, phone=9731423166, since=Fri Sep 02 13:02:46 UTC 2022
```

Figure 4: Audit log

```
metadata:
  labels:
    app: ums-journal-service
spec:
  containers:
  - name: ums-journal-service
    image: glarimy/ums-journal-service
```

Create a new deployment on top of the other deployments we made earlier.

```
kubectl create -f <deployment-file-name.yml>
```

Check the pods in the cluster. It looks something like what's shown in Figure 1. You can observe that Kafka, MySQL, *AddService*, *FindService*, *SearchService* and *JournalService* have their pods running on the cluster.

A quick look at the service listing gives us the IP addresses exposed within the cluster, as in Figure 2. For example, the *AddService* is available on 10.109.157.19:8080.

Make a REST call using any suitable client like cURL to add a new user to the UMS. An example is given below:

```
curl -X POST -H 'Content-Type: application/json' -i
http://10.109.157.19:8080/user --data '{"name":"Krishna Mohan Koyya", "phone":9731423166}'
```

This call goes well, as shown in Figure 3.

Now, let us check if the *audits.log* is updated with the newly added user. For that, we need to get into the pod on

which the *JournalService* is running. From Figure 1, we know the pod-id. It is *ums-journal-service-5f7f8fb4d5-4tm4g*.

Use the following command to get into the pod:

```
kubectl exec -it ums-journal-service-5f7f8fb4d5-4tm4g - bash
```

Once the prompt is available, use the following command to peek into the *audits.log*:

```
cat audits.log
```

The result will be similar to Figure 4.

We can now celebrate our accomplishments. We are able to deploy a few desperate microservices, scale them individually, and make them collaborate. This is not trivial.

At the same time, it is not sufficient also.

As long as the end user is also on the same Kubernetes cluster, the UMS is available for adding, finding or searching the users. In other words, the end users should be on the same network of the cluster in the data centre. You will agree with me that this doesn't make any sense!

The end users are always on a public network. They cannot and should not be on the data centre network. So, how is the end user able to access the UMS?

This is where the *API gateway* pattern plays an important role. A *gateway* acts as the entry point into the microservice infrastructure. It runs on a public IP address so that the end users can connect to it. The *gateway*, upon receiving the requests from the end users, needs to delegate them to the appropriate microservices.

Figure 5 shows what such a setup looks like.